WordPress Day 2 – Assignment – Sandhya Shiwakoti

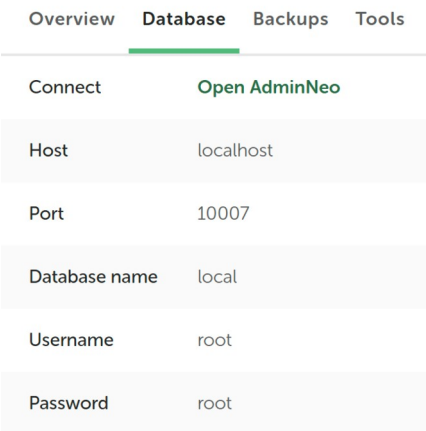
Task 1: Explore WP Local's

a. Explore the Site's Database Access

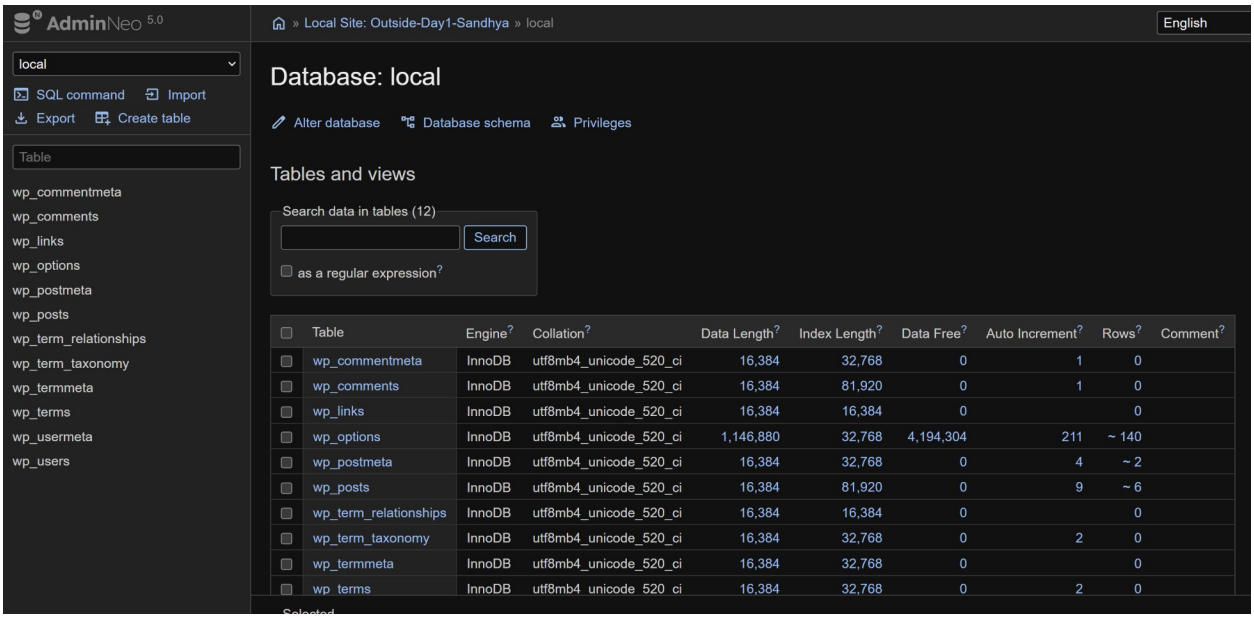
LocalWP gives direct access to the WordPress database through a built-in Admin interface. This is useful for viewing tables, running queries or checking data without needing a separate database tool.

Step 1: Open LocalWP and start site.

Step 2: Click on the Database tab in the site panel.

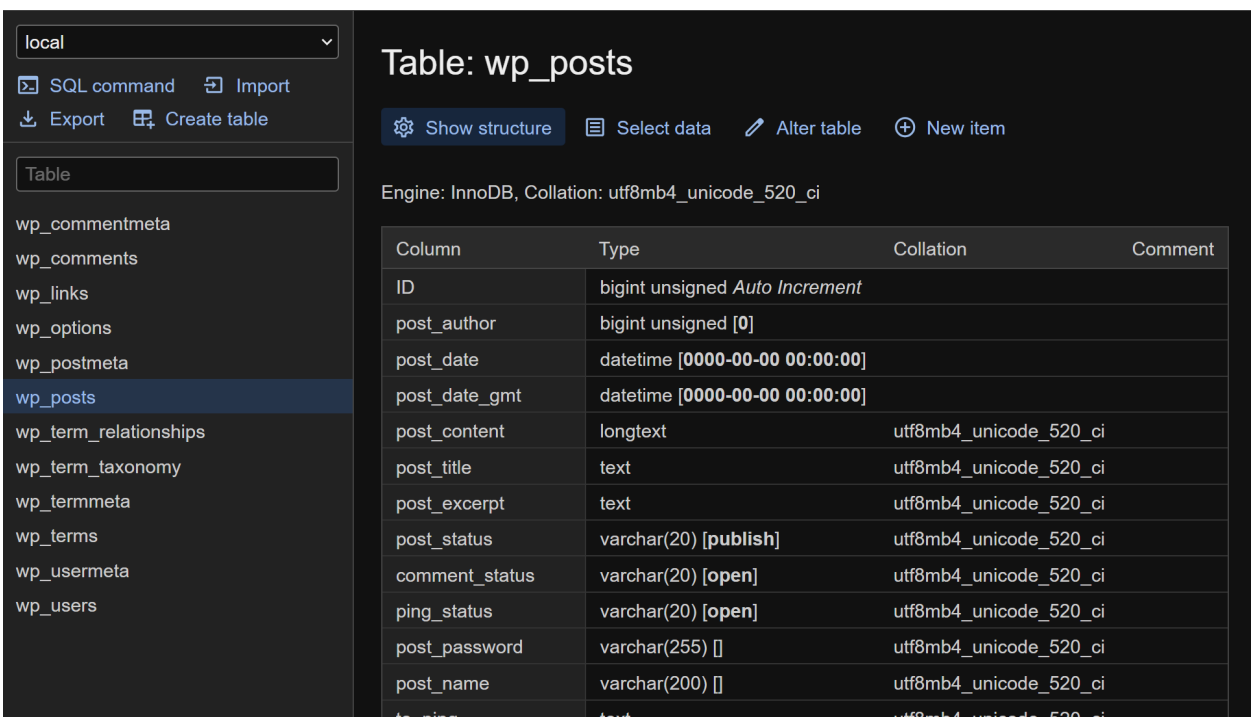


Step 3: Click Open AdminNeo. A browser tab will open showing the database.



Step 4: Explore tables on the left side. Key ones:

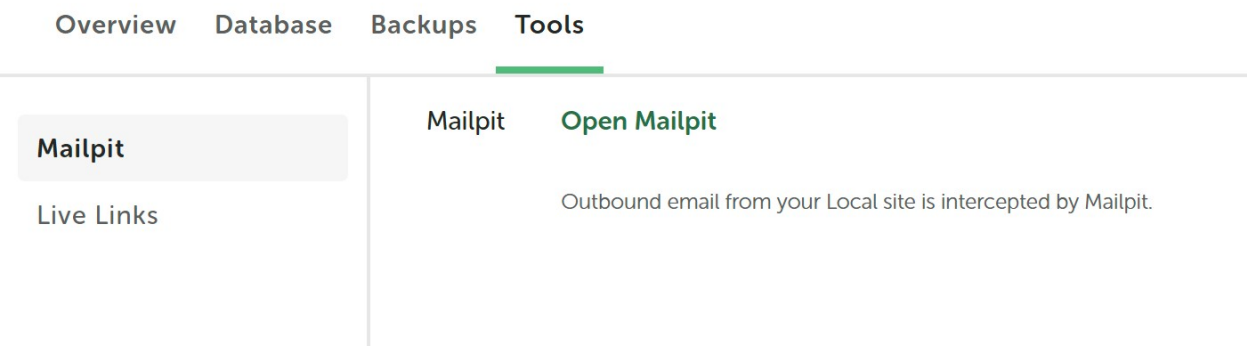
wp_posts (all content), wp_users (user accounts), wp_options (site settings), and wp_postmeta (extra data attached to posts).



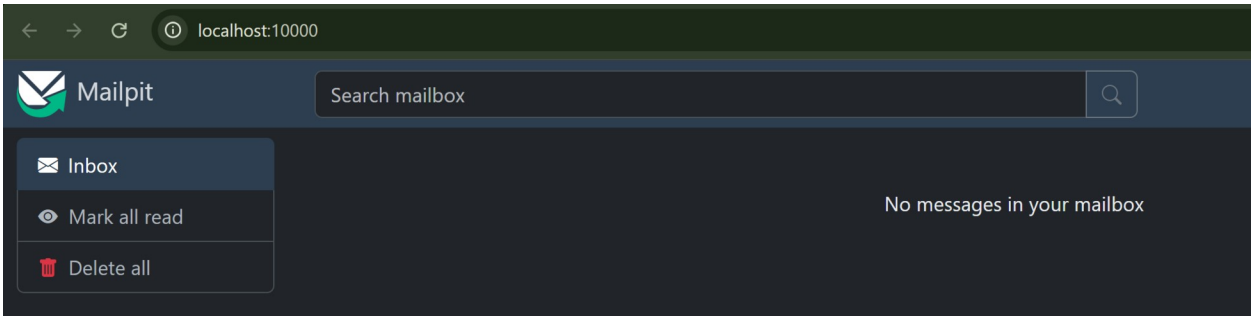
b. Explore Mailpit

Mailpit is a local email testing tool built into LocalWP. It stores emails sent by WordPress on our local computer instead of sending them to real email addresses. This lets us view and test emails locally.

Step 1: Go to tools tab in the LocalWP site panel.

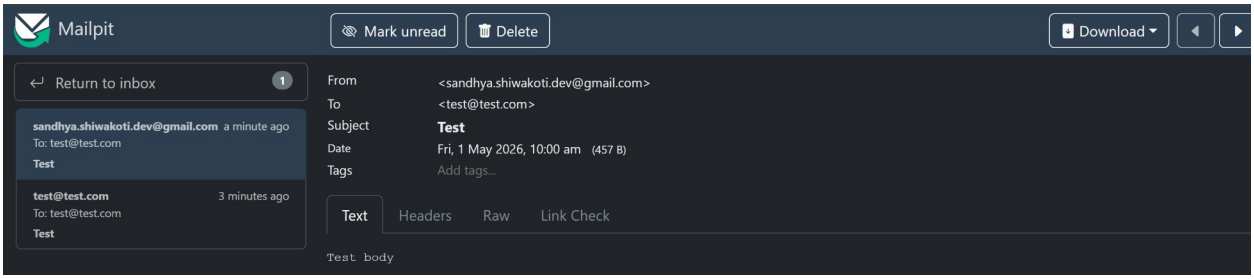


Step 2: Click Open Mailpit. It will open in a browser tab.



Step 3: Go to wp-login.php and try Lost Password feature from the login page. Enter admin email and submit.

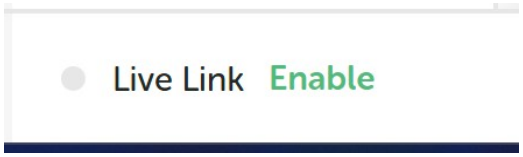
Step 5: Go back to Mailpit and we should see the password reset email appear there. Click on it to see the full email contents.



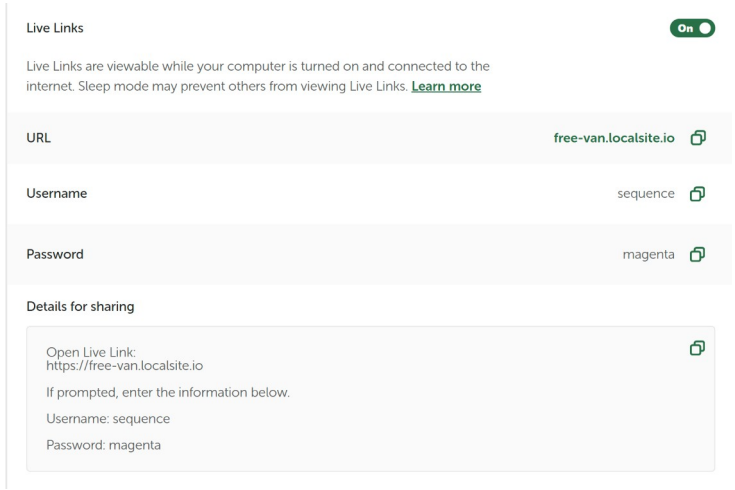
c. Live Links

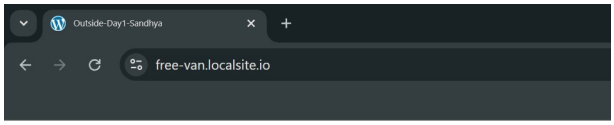
Live Links lets us share local WordPress site with someone else over the internet temporarily. This is helpful when we want to show work to a client or colleague without deploying the site.

Step 1: At the bottom of the site, we will see Live Link with an Enable button next to it. Click Enable.



Step 3: LocalWP will generate a public URL. Anyone with this link can view our site in their browser.





Outside-Day1-Sandhya

Sample PageTeam

Hello world!

Posted on April 25, 2026 by sandhya

Welcome to WordPress. This is your first post. Edit or delete it, then start writing!

Step 4: To stop sharing, click Disable on the same Live Link option.

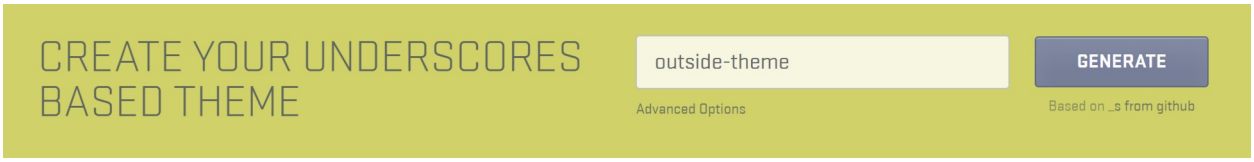
Task 2: Explore Theme Development using underscores.me

Underscores is a starter theme for WordPress. It gives us clean, minimal base with the right file structure already so we can build a custom theme on top of it.

Getting the Theme:

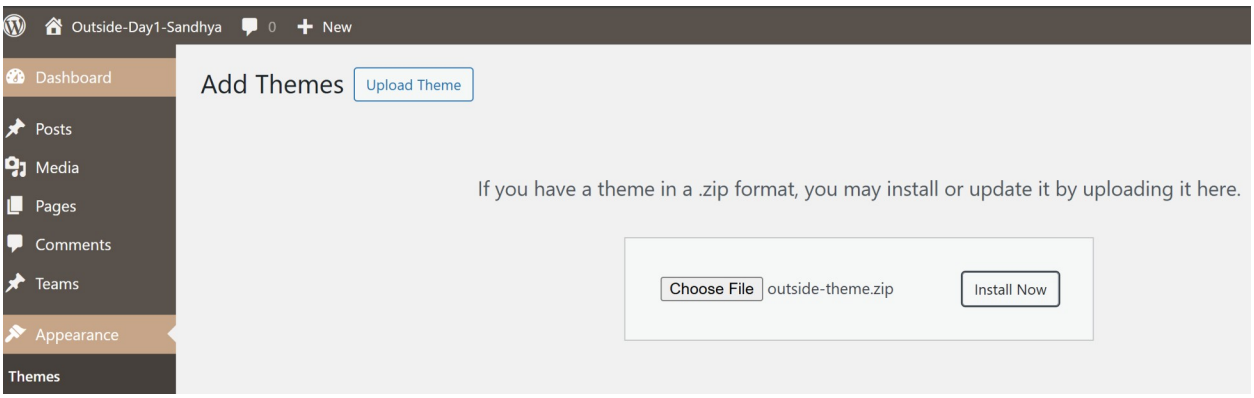
Step 1: Go to underscores.me in browser.

Step 2: Enter theme name (for example: outside-theme). It will download a .zip file.



Step 3: Go to WordPress admin dashboard.

Go to Appearance → Themes → Add New → Upload Theme. Upload the .zip file and activate it.



a. Create a Custom Post Type - Team

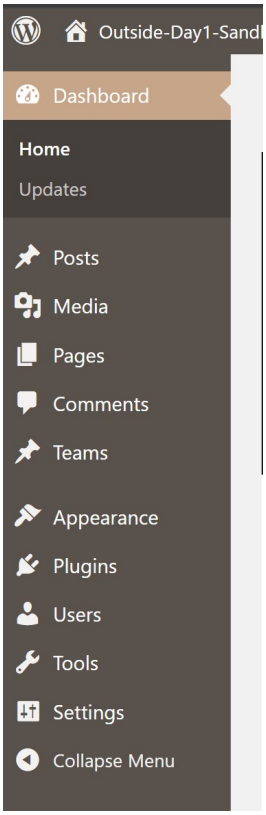
A Custom Post Type (CPT) lets us create a new type of content in WordPress beyond the default Posts and Pages. Here we create one called team to manage team member profiles.

Step 1: Open VS Code from LocalWP.

Step 2: Navigate to app/public/wp-content/themes/outside-theme/ and open the functions.php file.

Step 3: At the bottom of functions.php, add the following code:

Step 4: Save the file. Go back to WordPress dashboard and we should see a new Teams menu item appear in the left sidebar.



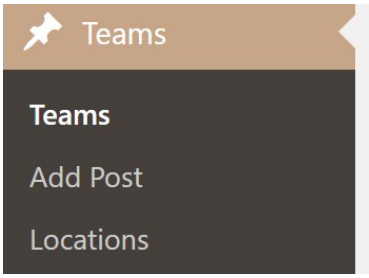
b. Create a Custom Taxonomy - Location

A custom taxonomy works like categories but for our custom post type. Here we create a location taxonomy so each team member can be assigned to a location.

Step 1: In the same functions.php file, add this code below the previous function:

```php
function create_team_taxonomies() {
    $labels = array(
        'name' => 'Locations',
        'singular_name' => 'Location',
        'menu_name' => 'Locations',
    );
    register_taxonomy(
        'location',
        array( 'team' ),
        array(
            'hierarchical' => true,
            'labels' => $labels,
            'rewrite' => array( 'slug' => 'location' ),
        )
    );
}
add_action( 'init', 'create_team_taxonomies', 0 );
```

Step 2: Save the file. Go to dashboard and click on Teams. We should now see a Locations submenu under Teams where we can add location terms.



c. Add Metabox and Custom Fields for Position and Years of Experience

A metabox adds an extra section to the post edit screen. We use it here to add custom fields for storing a team member's position and years of experience.

Step 1: In functions.php, add the following two functions. The first one registers the metabox, and the second one displays the fields inside it:

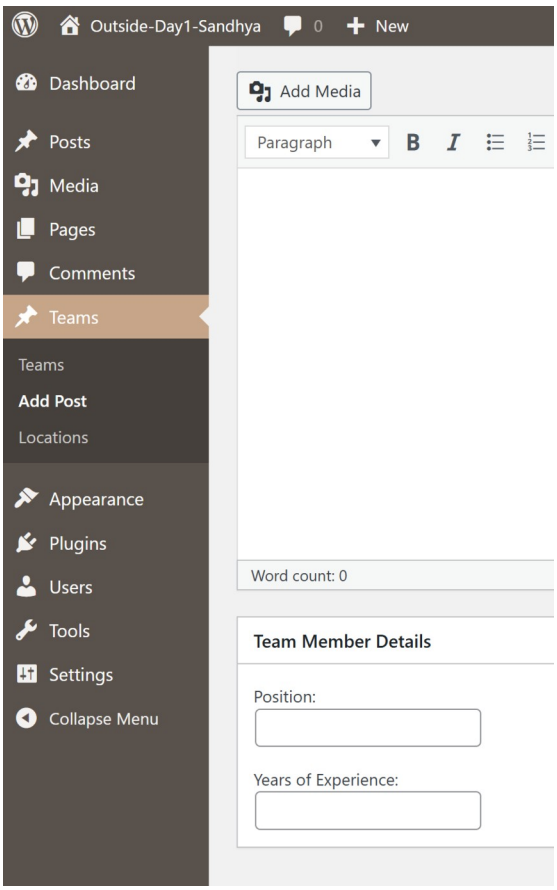
```php
function team_metabox() {
    add_meta_box(
        'team_metabox',
        'Team Member Details',
        'team_metabox_callback',
        'team',
        'normal',
        'default'
    );
}
add_action( 'add_meta_boxes', 'team_metabox' );

function team_metabox_callback( $post ) {
    wp_nonce_field( basename( __FILE__ ), 'team_nonce' );
    $position = get_post_meta( $post->ID, 'team_position', true );
    $years = get_post_meta( $post->ID, 'team_years', true );
    ?>
    <p>
        <label for="team_position">Position:</label><br/>
        <input type="text" name="team_position" id="team_position"
        value="<?php echo esc_attr( $position ); ?>" />
    </p>
    <p>
        <label for="team_years">Years of Experience:</label><br/>
        <input type="number" name="team_years" id="team_years"
        value="<?php echo esc_attr( $years ); ?>" />
    </p>
    <?php
}
```

Step 2: Now add a save function so the field values actually get stored when the post is saved:

```
function team_save_meta( $post_id ) {
    if ( ! isset( $_POST['team_nonce'] ) ) return;
    if ( ! wp_verify_nonce( $_POST['team_nonce'], basename( __FILE__ ) ) ) return;
    if ( defined( 'DOING_AUTOSAVE' ) && DOING_AUTOSAVE ) return;
    if ( isset( $_POST['team_position'] ) ) {
        update_post_meta( $post_id, 'team_position',
        sanitize_text_field( $_POST['team_position'] ) );
    }
    if ( isset( $_POST['team_years'] ) ) {
        update_post_meta( $post_id, 'team_years',
        sanitize_text_field( $_POST['team_years'] ) );
    }
}
add_action( 'save_post', 'team_save_meta' );
```

Step 3: Save functions.php. Go to Teams → Add New in the dashboard. We now see a Team Member Details box below the editor with fields for Position and Years of Experience.



Task 3: Explore Theme Development using Sage

Sage is a WordPress starter theme by Roots. It is used by offices for real projects. Underscores uses plain PHP while Sage uses modern tools like Blade templates (from Laravel), Tailwind CSS, and npm for asset management. It is more advanced and provides cleaner and organized development workflow.

Exploration:

Step 1: To install Sage, open site's terminal in LocalWP by clicking Site Shell and run:

```
C:\Users\ACER\Local Sites\outside-day1-sandhya\app\public>composer create-project roots/sage outside-theme
Creating a "roots/sage" project at "./outside-theme"
```

Step 2: Navigate into the theme folder and install dependencies by running:

```
C:\Users\ACER\Local Sites\outside-day1-sandhya\app\public>cd wp-content\themes\sage-theme

C:\Users\ACER\Local Sites\outside-day1-sandhya\app\public\wp-content\themes\sage-theme>npm install
```

Step 3: Start the development server by running:

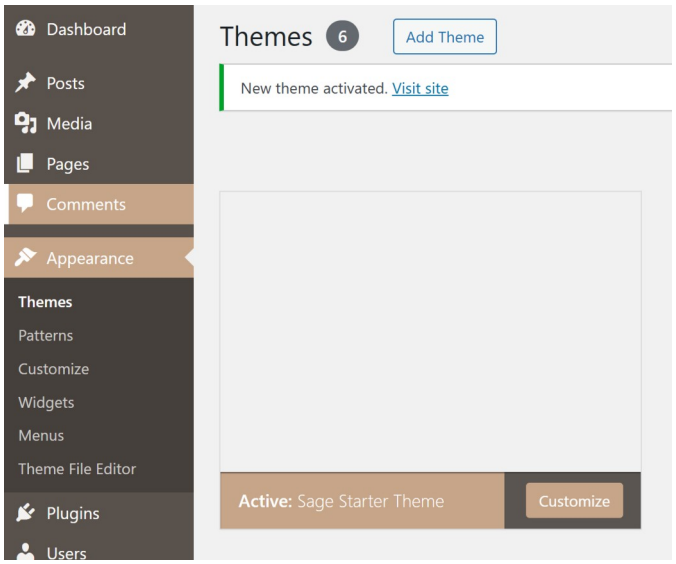
```
ACER@DESKTOP-R7I0FQN MINGW64 ~/Local Sites/outside-day1-sandhya/app/public/wp-content/themes/sage-theme
$ npm run dev
> dev
> vite


  VITE v8.0.10  ready in 1112 ms

  ➜  Local:   http://localhost:5173/app/themes/sage/public/build/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help

  LARAVEL   plugin v3.1.0
```

Step 4: Go to your WordPress dashboard → Appearance → Themes and activate the sage-theme.

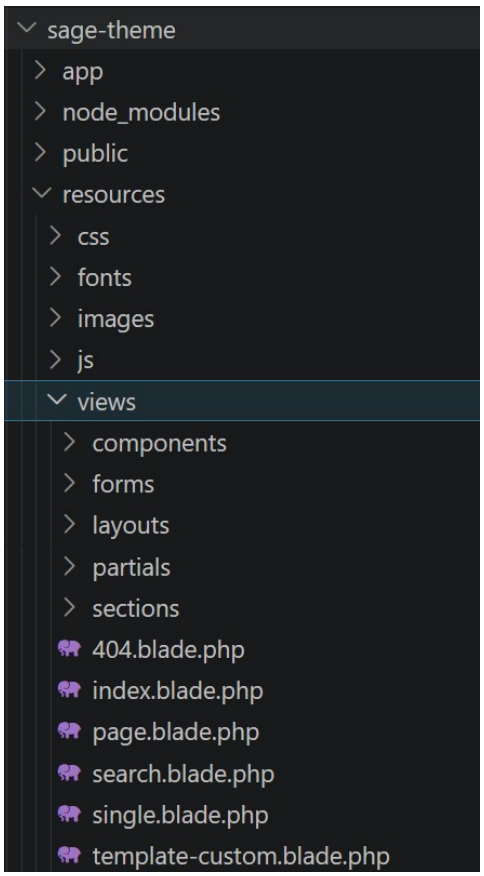


Step 5: Explore the folder structure inside sage-theme. Look at these folders:

resources/views/ - this is where the Blade template files (.blade.php) are.

app/ - handles logic like menus and assets

public/ - compiled CSS and JS files go here after build



Step 6: Open .blade.php file inside resources/views/ . We can see that the syntax is so clean compared to regular PHP templates, we can see @if, @foreach, @include etc.

wp-content > themes > sage-theme > resources > views > index.blade.php

```
@extends('layouts.app')

@section('content')
  @include('partials.page-header')

  @if (! have_posts())
    <x-alert type="warning">
      {!! __('Sorry, no results were found.', 'sage') !!}
    </x-alert>

    {!! get_search_form(false) !!}
  @endif

  @while(have_posts()) @php(the_post())
    @includeFirst(['partials.content-' . get_post_type(), 'partials.content'])
  @endwhile

  {!! get_the_posts_navigation() !!}
@endsection

@section('sidebar')
  @include('sections.sidebar')
@endsection

```